BioHackathon series:
DBCLS BioHackathon 2025
Mie, Japan, 2025
MIE redesign

Submitted: 26 Jul 2026

Published by BioHackrXiv.org

Measure before you rewrite: ablation-driven redesign of LLM-facing RDF schema documentation in TogoMCP

Akira R. Kinjo [1] **and Yasunori Yamamoto** [2]

1 Anima Machina G.K., Osaka Station Building No. 3, 29th Floor, 1-1-3 Umeda, Kita-ku, Osaka 530-0001, Japan **2** Database Center for Life Science, Joint Support-Center for Data Science Research, Research Organization of Information and Systems, 10-3 Midori-cho, Tachikawa, Tokyo 190-0014, Japan

Abstract

MIE files are per-database YAML documents that TogoMCP supplies to a large language model at query time so it can compose SPARQL against the DBCLS RDF Portal. Ours had grown to eleven sections, semi-automatically generated for each of 36 databases and reviewed by hand. Each section had been introduced in response to a systematic query failure observed in use — sound practice, but it left open whether any section still earned its tokens once the other ten were present. We measured that, in eighteen ablation conditions across four families: is a section necessary, is a functional group necessary, is the whole document worth anything, and is any one group sufficient alone. No single section and no single group is necessary. Removing the entire document costs 0.9 points out of 20, and the query-construction group alone recovers 99% of that — the whole is worth roughly 2.7 times the sum of its parts, the signature of heavy redundancy. We rebuilt the format around that evidence, making the verified executable example the atomic unit: 36 files, 303 examples, each 29–65% smaller than the file it replaces. A pre-registered equivalence run over 100 benchmark questions finds v3 statistically indistinguishable from v2 in answer quality (+0.29/20, 95% CI [-0.09, +0.68]) while using 15% fewer input tokens, costing 15% less and running 6% faster, with the factoid-question score up a full point. We also report eight measurement traps that faked or destroyed signal, and one budgeting error worth more than the results: we spent the most on the least informative experiment, and say what we would do instead.

Introduction

TogoMCP exposes the RDF Portal knowledge graph maintained by DBCLS (Kawashima et al., 2018) — which aggregates roughly 60 life-science databases — to large language models through the Model Context Protocol (Anthropic, 2025), so that a researcher can ask a biological question in natural language and have an agent compose, execute, and interpret the SPARQL that answers it (Kinjo et al., 2026). The mechanism that makes this work is not the model's SPARQL fluency but the **MIE** (Metadata-Interoperability-Exchange) file: a per-database YAML document, delivered to the model at query time, that supplies the schema-level context the model cannot invent — the non-obvious predicates, the join paths, the count and graph traps, and worked example queries. Supplying schema context at query time rather than relying on the model's own recall is the design other recent text-to-SPARQL systems converge on as well (Emonet et al., 2024; Zahera et al., 2024).

An MIE file is, in other words, documentation written for a machine reader — and largely written *by* one: the files are produced semi-automatically, by a pipeline of automated schema discovery, LLM-assisted drafting, validation against the live endpoint, and expert review, at roughly two to four hours of expert effort per database (Kinjo et al., 2026). What that pipeline emits is fixed by the specification, which had grown to eleven sections.

Those sections were not guesses. Each was introduced in response to a *systematic failure* observed in use: a class of SPARQL query that kept going wrong the same way, documented

so it would stop going wrong. That is a sound way to build documentation, and it is the reason the corpus works. What it does not establish is whether any given section still earns its tokens once the other ten are present — because a failure mode can be fixed in more than one place, and eleven rounds of locally-correct fixes need not add up to a well-proportioned document. Nobody had checked.

The checking matters because the cost is recurring. Every byte of an MIE file is re-read on every turn of every session and multiplied across the whole corpus, and the token budget spent on documentation is a budget not spent on reasoning or on query results. Stated generally: **what belongs in a "schema card" for an LLM agent, and how would you know?** Anyone building an MCP server over a structured resource — a database, an API, a filesystem — accumulates an answer the way we did, one observed failure at a time, and rarely gets any signal about the result.

We set out to answer it empirically for TogoMCP, and this report is about the answer and the way we got it. Our contributions are four:

1. **A reusable ablation harness and statistical protocol** for LLM-facing documentation, built around four question types — is a part *necessary*, is a functional group *necessary*, is the whole thing worth anything, and is any one group *sufficient* — together with a corrected recommendation about which order to ask them in, since the order we chose spent the most on the least.
2. **The finding**: the value of our documentation is real but heavily redundant, and it is concentrated almost entirely in query-construction content. Removing any single section, or any whole functional group, costs nothing measurable. Removing all of it costs ~0.9 points out of 20. One group alone recovers 99% of that.
3. **MIE v3**, a format derived from that evidence rather than from taste, built around the verified executable example as the atomic unit of documentation.
4. **A pre-registered equivalence release**: at n=100 questions, v3 is statistically indistinguishable from v2 in answer quality while consuming **15% fewer input tokens**, costing **15% less**, and running **6% faster** — with the factoid-question judge score up by a full point.

It is worth saying up front what we did *not* find, because most of our measurements are null and those nulls are the result. Sixteen of our eighteen ablation conditions produced no statistically significant effect on answer quality. Read individually, each null invites the conclusion that the content was worthless. Read as a set, alongside the two conditions that *did* move, they say something quite different — and the difference is the methodological heart of this report.

TogoMCP, MIE files, and the benchmark

TogoMCP is a FastMCP server that mounts several sub-servers behind a common surface. Its two load-bearing tools are `run_sparql`, which executes a query against a named RDF database, and `get_MIE_file`, which returns that database's MIE document. Around them sit REST wrappers (UniProt, ChEMBL, PDB, PubChem, Reactome, Rhea, MeSH) and identifier services (TogoID (Ikeda et al., 2022), NCBI E-utilities, TogoVar). The published description of the system and its original 50-question evaluation is in *Database* (Kinjo et al., 2026); that work established that the tool-augmented system as a whole, MIE files included, substantially outperforms an unaided LLM. The present work asks a question interior to the system: given that it works, which parts of the documentation are carrying it?

A note on the word "baseline," because the two studies use it for opposite things. In the earlier work, the baseline is the *unaided* LLM — no tools at all — and every condition is scored by how far it improves on that. In this report, the baseline is the *fully equipped* system: TogoMCP serving the complete MIE corpus, which is the best configuration available rather than the worst, and every ablation is scored by how much removing something costs relative to it. The earlier design measures the distance from nothing to something; ours measures

the dent made by taking a piece out of something that already works. A consequence worth keeping in view is that our effects are bounded by a much narrower range — the earlier study had roughly 3.5 points of headroom over its baseline, whereas ours starts at 17/20 and the entire document is worth 0.9 of the remaining 3.

TogoMCP currently documents **36 of the portal's databases, one MIE file each** — and those 36 are served by only **ten distinct SPARQL endpoints**. Sixteen of them share a single endpoint. That many-to-one structure is not incidental to this report: a query that fails to scope itself to the right named graph silently ranges over its neighbours, so co-tenancy is a recurring source of count inflation and one of the things an MIE file exists to warn about. It is why the v3 header carries an explicit `co_hosted` flag for every dataset sharing an endpoint.

How those files come into being matters for what follows. Each is generated by a four-phase pipeline — schema discovery by a prescribed series of exploratory SPARQL queries; LLM-assisted drafting into the specified sections; validation of every example against the live endpoint; and a final expert review for domain accuracy — implemented as an agent skill and costing roughly two to four hours of expert effort per database. Two consequences follow. The specification is not merely a style guide but a *generation target*: whatever sections it names are what gets produced, for every database. And because a human reviews each file, the corpus is expensive enough that nobody rewrites it casually. Changing the format is therefore a corpus-wide, 36-file, expert-reviewed commitment — which is precisely why we wanted evidence before making one.

The format has a BioHackathon lineage of its own. It was largely formalized at the DBCLS BioHackathon 2025 in Mie Prefecture, within a project on MCP server tools backed by RDF shapes (Labra-Gayo et al., 2025), and takes its name from that prefecture; the backronym *Metadata-Interoperability-Exchange* was suggested by Jose-Emilio Labra Gayo. The measurement and redesign reported here were carried out a year later at **Togothon**, the monthly knowledge-graph meeting DBCLS has run since its SPARQLthon days. So this is a hackathon artifact revised at a hackathon — which is perhaps the natural life cycle for this kind of thing, and an argument for the smaller recurring meeting as the place where the unglamorous follow-up work actually gets done.

Version 2.3 of the MIE format specified eleven sections. For the ablations we assigned each to one of three functional groups, which turned out to be the more useful unit of analysis.

Table 1: The eleven sections of the MIE v2.3 format, grouped by function, with each group's share of total corpus bytes.

Group	Sections	Share of MIE bytes
query	`schema_info`, `shape_expressions`, `sparql_query_examples`, `cross_references`, `cross_database_queries`	53%
guardrails	`critical_warnings`, `common_errors`, `anti_patterns`	25%
orientation	`architectural_notes`, `data_statistics`, `sample_rdf_entries`	22%

Inspecting the corpus by hand suggested a specific hypothesis, which the ablations were designed to test: **the same fact was routinely documented three times over, each time in a different form.** A single predicate might appear once as a ShEx shape in `shape_expressions` — stating it as a declarative constraint — once as a sample triple in `sample_rdf_entries`,

showing it as a concrete instance, and once inside a worked query in `sparql_query_examples`, using it as an executable step. Three modes of expression, one underlying fact, three separate sections; the `cross_references` list acted as a loose fourth restatement for cross-reference predicates. (`schema_info` is not one of these; it is the file's metadata header, and it survives into v3.) If that hypothesis were right, no single section would ever look necessary, because its siblings would cover for it.

The triplication is not hard to account for, given how the sections arrived. A newly observed failure — the model misusing some predicate — can be forestalled by tightening the shape, by adding a sample triple, or by supplying a worked query, and any of the three is a defensible response. Over eleven such rounds, spread across time and authors, the same underlying facts accumulate in several places at once. Every individual decision is locally correct; the aggregate is redundant. This is a property of incremental, failure-driven documentation in general, not a lapse peculiar to us, and it is exactly the property that a leave-one-out ablation cannot see.

Measurements are against an internal benchmark of **100 biologically grounded questions**, 20 in each of five types (`yes_no`, `factoid`, `list`, `summary`, `choice`), spanning 34 databases, with creation-time coverage targets of at least 60% requiring two or more databases and at least 20% requiring three or more. Questions were screened so that the answer is not recoverable from the published literature — live database access is necessary. Answers are scored by an LLM judge (Zheng et al., 2023) on four criteria (recall, precision, non-redundancy, readability) (Tsatsaronis et al., 2015), 1–5 each, for a total of **4–20**; a binary exact-answer grader runs alongside on the gradable subset. Answering used `claude-sonnet-4-5` throughout; judging used Claude Opus through the Anthropic API with forced tool use, pinned to `claude-opus-4-8` for the section sweep and left at the evaluation default for later sweeps.

Method: an ablation harness for documentation

The harness is simple in principle. A script generates a modified corpus — one section stripped from every file, or one group, or everything but one group — a local TogoMCP instance is pointed at that corpus via an environment variable, and the benchmark is run against it in isolated sessions with no conversation history. Each condition is compared against a baseline run in the same batch.

We ran four families of condition, in this order:

1. **Leave one section out** (11 conditions). Asks: is this section *necessary*, given that the other ten remain?
2. **Leave one group out** (3 conditions). Asks the same question with redundancy partly suppressed: if `shape_expressions` was covered for by `sparql_query_examples`, removing both at once should expose the loss.
3. **Remove the whole MIE** (1 condition). Asks: what is all of this worth? Implemented by blocking `get_MIE_file` at the tool level rather than by stripping the corpus, so the agent retains every other tool and loses only the documentation.
4. **Keep one group only** (3 conditions). Asks the complementary question: is any single group *sufficient* on its own?

Families 1 and 2 measure necessity; family 3 measures total value; family 4 measures sufficiency. **A null in family 1 or 2 is ambiguous between "worthless" and "redundant", and only families 3 and 4 can disambiguate it.**

We would not run them in this order again, and the reason is worth more than the results. Cost scales with the number of conditions, so the family that asks the narrowest question — eleven separate leave-one-outs — is also by far the most expensive. It consumed roughly US$845, more than families 3 and 4 combined (four conditions, about US$280), and it produced nothing that changed the design. The two cheap families produced both findings that did.

Worse, the section sweep was underpowered by construction, and knowably so. Its multiple-comparison bar is $|z| > 2.84$. Had we first learned what family 3 later told us — that the entire document is worth about 0.9 points out of 20 — we could have computed that value spread over eleven sections implies per-section effects near 0.08, which no achievable sample size would resolve on a benchmark whose baseline already sits at 17/20. The one experiment that could have set that expectation, whole-MIE removal, is a **single condition**: the cheapest thing we ran, and the only one whose result gates the interpretation of everything else.

So the ordering we would recommend is close to the reverse of the one we followed. Start with **total removal** — one condition, and if it is null you are done, because no decomposition can find value that the whole does not have. Use its effect size as the budget you are allocating. Then **leave-one-in at the group level**, which localizes that value cheaply; `keep_query` alone answered the design question. Only then consider **per-component leave-one-out**, and only if the total effect is large enough that the per-component share could clear a corrected threshold at an affordable n. Leave-one-out is the reflexive default in ablation work, and for documentation — where redundancy is expected, total effects are small, and judge scores saturate — it is close to the worst place to start.

Every effect reported below is a **paired per-question difference with a 95% confidence interval**, never an aggregate ratio (see Trap 2). We report trimmed analyses that exclude ceiling (20/20) and floor (<12/20) questions alongside untrimmed ones. Multiple-comparison thresholds are stated per family: $|z| > 2.84$ for the eleven sections, $|z| > 2.39$ for each three-condition family, and $|z| > 1.96$ for the single planned whole-MIE comparison.

To let others budget: each condition was 40 pilot questions $\times$ 3 replicates. The section sweep was 12 conditions — a baseline plus the eleven leave-one-outs — at roughly US$845 and 72 hours of wall clock; the group sweep, 4 conditions (a baseline plus three groups) at roughly US$265 and 27.5 hours. The whole-MIE and keep-one-group sweeps added four more conditions on top of that, pairing against the group sweep's baseline rather than re-running one of their own. This is not a cheap experiment, which is itself an argument for designing it carefully before starting.

Results I: the redundancy arc

No single section is necessary

Of the eleven sections, **zero** produced a confidence interval excluding zero — on the judge score, on exact-answer correctness, or on query effort. The baseline scored 17.13/20.

Table 2: Leave-one-section-out contributions to judge score (points out of 20), paired per question, n=34 after ceiling/floor exclusions. Positive means removing the section hurt.

Section	Contribution (±95% CI)	z
`common_errors`	+0.65 ± 0.65	+1.94
`cross_database_queries`	+0.25 ± 0.77	+0.63
`architectural_notes`	+0.23 ± 0.51	+0.91
`schema_info` †	+0.20 ± 0.50	+0.77
`critical_warnings`	+0.14 ± 0.62	+0.43
`data_statistics`	+0.07 ± 0.66	+0.21
`anti_patterns`	+0.04 ± 0.40	+0.19
`cross_references`	-0.06 ± 0.52	-0.22
`sample_rdf_entries`	-0.08 ± 0.63	-0.24
`sparql_query_examples`	-0.13 ± 0.57	-0.44
`shape_expressions`	-0.15 ± 0.54	-0.53

† Not a clean leave-one-out — see Trap 3.

`common_errors` is the one near-miss, at nominal p of about 0.052 and the only section pointing the same way on both quality and effort. **It is not a finding.** Eleven sections were tested; the corrected threshold is $|z| > 2.84$, and one borderline hit in eleven is roughly what chance produces. Scaling from its effect size, resolving it after correction would need n of about 73 — so the null is bounded, not merely asserted.

No single group is necessary either

Removing an entire functional group was also null on the judge score. Removing the **whole query group — 53% of the MIE by bytes — cost +0.20 ± 0.40 points**. This is a separate batch with its own in-batch baseline, 16.88/20 trimmed; the whole-MIE and keep-one-group conditions below pair against that same baseline rather than against the section sweep's 17.13. That figure needs the caveat attached to it rather than deferred: the query group contains `schema_info`, so this condition also broke the database-discovery tool, making it partly a discovery-breakage-then-recovery result rather than a clean measurement of query-construction content (Trap 3). The whole-MIE condition below, which leaves discovery working, is the clean read. The pre-registered prediction, formed by summing the single-section effects, was that `guardrails` would lead; it came last, at +0.04. The summation heuristic had no predictive value.

Exact-answer correctness produced one near-miss: dropping `query` cost +0.088 ± 0.087 ($z = 1.97$), whose untrimmed interval barely excludes zero at the single-comparison threshold but fails the k=3 correction and reverts to including zero when trimmed. A real trend, still underpowered.

At this point in the investigation, the redundancy hypothesis looked refuted. If siblings were covering for each other, removing a whole group should have exposed the loss, and it did not.

Removing everything is significant

The escalation reversed that reading. Blocking `get_MIE_file` entirely cost **+0.93 ± 0.68 ($z = 2.68$)** on the judge score, stable across judging treatments (+0.88 ± 0.66 with five judges; +0.91 ± 0.72 trimmed), p of about 0.007–0.02 against the $|z| > 1.96$ bar for a single planned comparison. Validity was confirmed server-side: zero `get_MIE_file` executions, with 13 blocked attempts across the condition — the model still reflexively reached for it.

One caveat by our own Trap 1: this condition pairs against the group sweep's baseline rather than against one re-run beside it. The effect is large against that baseline's own replicate drift of ±0.35, so it survives a worst-case baseline shift — but a fresh in-batch baseline would have been the better design, and we say so having just made the rule.

Table 3: The redundancy arc. The whole is worth roughly 2.7× the sum of its parts. The section and group rows are trimmed analyses; the whole-MIE row is untrimmed (+0.91 ± 0.72 trimmed).

Removed	Contribution	Significant?
one section (x 11)	at most +0.65	no
one group (x 3)	at most +0.20	no
Sum of the three groups	+0.34	—
the whole MIE	**+0.88 to +0.93**	**yes**

Super-additivity of this size is the signature of strong redundancy, and it disambiguates the nulls. Alone, the group results meant "no value **or** redundant." With the whole-MIE result in hand they mean **redundant**: the content genuinely helps, and no individual part of it is load-bearing because the others cover.

One group is sufficient

The sufficiency complement completes the picture. Each `keep_<group>` condition retains only that group and strips the other two; sufficiency is measured against the no-MIE condition.

Table 4: Leave-one-in (sufficiency), untrimmed. "% gap" is the share of the +0.93 whole-MIE effect recovered; "complement" (baseline - keep) is what removing the *other* two groups costs. An asterisk marks a confidence interval excluding zero.

Group kept	Sufficiency (± 95% CI)	z	% gap	Complement
query	**+0.92 ± 0.54 ***	**+3.32**	**99%**	+0.01 ± 0.54
orientation	+0.41 ± 0.89	+0.89	44%	+0.52 ± 0.78
guardrails	+0.12 ± 0.66	+0.36	13%	+0.81 ± 0.64

The query group **alone** — schema, shapes, examples, cross-references, cross-database queries — recovers 99% of what the entire MIE provides, and its complement is +0.01: dropping guardrails and orientation from the full file costs nothing measurable.

One reassurance and one qualification keep this honest. The reassurance: the `keep_query` result is itself unconfounded, because every comparison it enters — sufficiency against no-MIE, complement against baseline — is between conditions that retain `schema_info`, so discovery is held constant throughout (Trap 3). The qualification: the other two rows *are* confounded, and in the direction that flatters the headline. Broken discovery handicaps them by up to about 0.2 points, so corrected sufficiency is nearer +0.61 for orientation and +0.32 for guardrails. Both remain non-significant and the story is unchanged, but the true query-versus-rest gradient is somewhat less extreme than the raw 99%/44%/13% suggests.

Figure 1 puts all eighteen conditions on one axis, which is where the argument becomes visible at a glance.

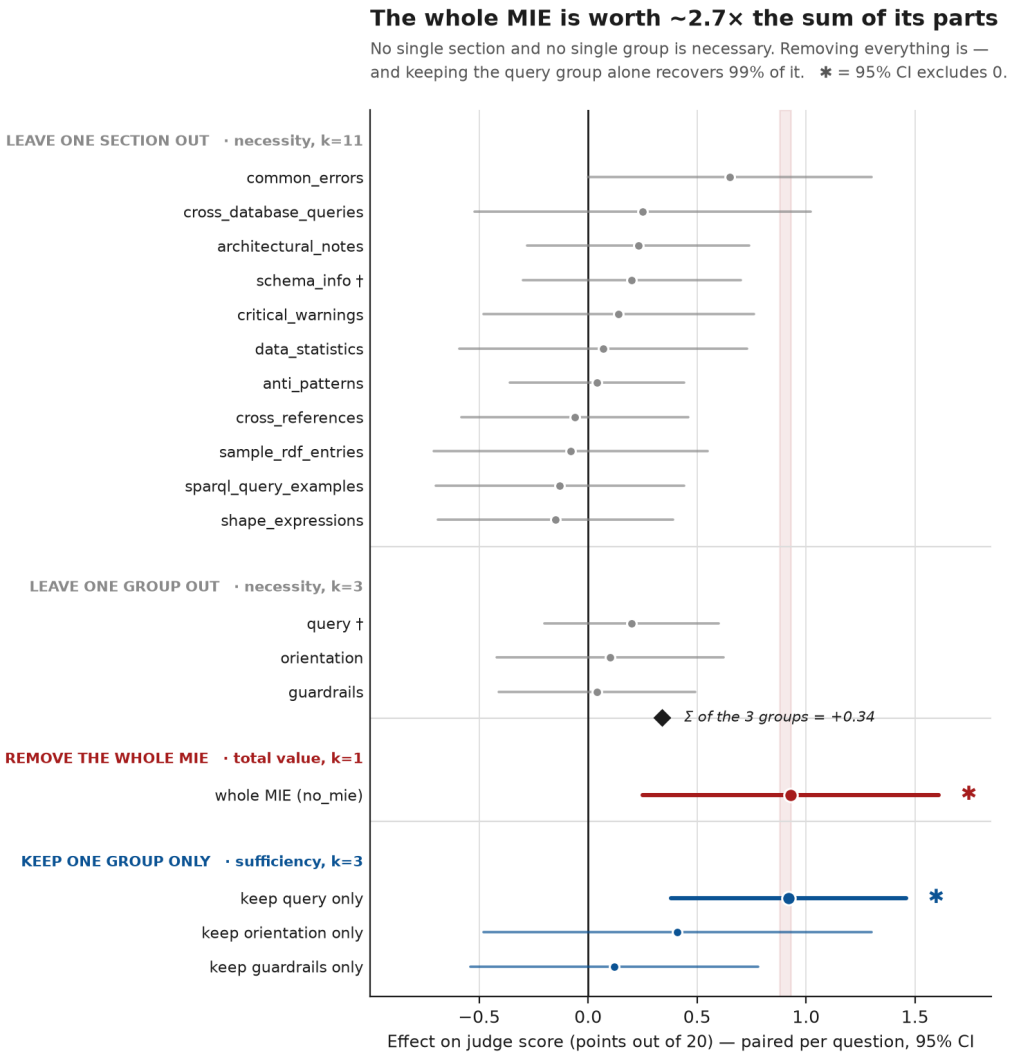

Figure 1: The redundancy arc. Eleven sections, three groups, the whole MIE, and three sufficiency conditions, all as paired per-question effects on judge score with 95% confidence intervals. Necessity is null everywhere; total value and the sufficiency of the query group are not. The shaded band marks the whole-MIE effect, +0.88 to +0.93. The daggered rows are not clean leave-one-outs: dropping `schema_info` — and with it the query group that contains it — also disables the `find_databases` discovery tool, making both dual ablations (Trap 3). Rows are comparable within a block but not across blocks: the two necessity blocks pair against their own baselines (17.13/20 for the sections, a separate in-batch 16.88/20 for the groups and the whole MIE), while sufficiency is measured against the no-MIE condition rather than against a baseline. The section and group rows are trimmed analyses; the whole-MIE and sufficiency rows are untrimmed (the whole MIE is +0.91 ± 0.72 trimmed), so the sum-versus-whole comparison in the panel title spans both treatments.

Two secondary findings

The one robust behavioural effect is not about quality. Removing `guardrails` *reduced* the number of SPARQL calls by -0.92 ± 0.92 per question, consistently across trimmed and untrimmed analyses. The interval excludes zero only narrowly — at the precision printed here it touches it, and the significance rests on the unrounded values — but the direction is stable and the interpretation is clear: the warnings provoke defensive querying. Whether that is a cost or a benefit depends on what one is optimizing; it is certainly not what the warnings were written to do.

Variance is answer-limited, not judge-limited. Decomposing the baseline's three answers × five judges gives judge-jitter SD 0.41 against between-answer SD 1.20; the per-question mean variance works out to 0.478 from the answer side and 0.011 from the judge side, i.e. **98% of it is agent stochasticity**. This reversed a standing assumption in our own notes. Re-judging is a cheap robustness check — it confirmed the whole-MIE effect is not judge noise — but it is not a power lever. The levers are more answer replicates and more questions.

Results II: traps that faked or destroyed signal

A substantial share of the project's elapsed time went into detecting and repairing measurement artifacts rather than into the measurements themselves — one harness bug alone cost nine days. These eight are the ones we would want to have been told about.

1. Never bank a baseline across batches. Our first analysis showed *every* section contribution slightly negative — removing anything apparently helped. The cause was that the baseline had been reused from a trial run one to two days earlier. Because all eleven contributions subtract the *same* baseline, they are **one event, not eleven**: a baseline sitting 0.4 low drags them all negative together. Re-running the baseline fresh in the same batch moved it from 16.72 to 17.13 and the uniform pattern dissolved into an unremarkable seven-positive, four-negative spread. Resume logic that skips already-completed conditions is convenient and dangerous for exactly this reason.

2. An aggregate difference is not a per-question effect. Removing `sparql_query_examples` drove SPARQL calls from 466 to 585 across the run — a 25% increase, and initially our clearest result. It does not survive pairing: per-question counts vary enormously (paired SD of about 3.2), and the paired delta was $+0.87 \pm 1.07$, comfortably including zero. The ratio was reading a sum as an effect.

3. An ablation can be a dual ablation. Stripping `schema_info` also broke `find_databases`, which builds its searchable catalog from that block. Its near-zero contribution is therefore a robustness result, not evidence the text is worthless. The tell is in the logs: fallback `list_databases` calls jump from 1–2 to 38 as the agent routes around a dead discovery front door. Two consequences follow. The `query` group inherits the same confound, since `schema_info` sits inside it. But by good luck of where it sits, the `keep_query` headline is measured *entirely between conditions that retain it* — sufficiency against no-MIE, complement against baseline — so discovery is held constant and the confound cancels exactly where it would have mattered most.

4. Your benchmark may leak through the tool schema. The complete 36-database roster appears in the `database` parameter's enumeration on `run_sparql` and `get_MIE_file`, and tool schemas are not stripped by any corpus ablation. Every condition therefore sees every database name. Our benchmark is *name-free but domain-transparent*: questions do not name their target database, but it is usually recoverable from the ever-present roster plus the model's priors. This bounds what any discovery-side ablation can measure, and it is the kind of leak that is invisible unless you go looking.

5. Gate on evidence that the condition actually ran. A relative `--results-dir` argument resolved against the wrong working directory, so rendered config paths silently failed to resolve, the runner fell back to production defaults, and **every group sweep for nine days served identical full MIEs** — zero ablation signal, no error message, plausible-looking numbers. The tell-tale was that a compromised condition's server log contained zero tool-call requests where a valid one contained about 1,500. We now assert non-zero tool calls per condition before trusting any result.

6. Subscription authentication cannot sustain a batch this size. Rate limiting produced `"Not logged in"` answer stubs that the runner recorded as successes, scoring 4/20 — entire conditions of 40 questions consisting of login errors. Fixed by forcing API authentication and

adding an abort-after-three-consecutive-failures guard so a throttled judge can never again silently score a run at the floor.

7. Token accounting must include the prompt-cache buckets. We recorded only the `input_tokens` field and dropped `cache_creation_input_tokens` and `cache_read_input_tokens` — which is exactly where a large document read is billed. A single-turn probe makes the scale vivid: `input_tokens: 10` against `cache_creation_input_tokens: 20,366` on the same call. **The redesign's own target metric was structurally unmeasurable** until this was fixed. The same fix removed a latent double-count that the under-reporting had been masking.

8. Spurious content-policy refusals contaminate agent benchmarks. 6.3% of cells returned a near-identical refusal message on benign microbiology questions — 7.0% of the v2 arm against 5.7% of v3, so not quite the balanced split it looks like in aggregate. Four questions measure nothing at all: two were refused on every replicate of *both* arms, and two more on every replicate of one arm, which leaves the pair with no comparable cell. Because a refusal floors a cell at $4/20$, an imbalanced split distorts a delta badly: one batch's raw $+1.31$ fell to $\mathbf{+0.58}$ once refusals were excluded — a genuine edge survived, but more than half the apparent effect was refusal luck rather than capability. At the level of a single question the distortion can account for the entire apparent result: one question's -5.3 vanished completely once its refused cells were dropped. Detect them by signature, report raw and clean, and trust the clean number.

Underlying all of these is a ceiling problem. With a baseline of 16.7–17.1 out of 20 and a per-question SD near 1, the judge score simply cannot resolve effects smaller than about 0.4 points at this sample size. Every null above should be read with that limit in mind.

The v3 redesign

The evidence points somewhere specific: the value is real, it is concentrated in query-construction content, and the format's organization by *authoring function* was scattering one fact across three sections. So we reorganized by **agent need × recoverability**, and made the **verified, executable worked example the atomic unit**. One example *is* the shape, the sample triple, and the annotated warning that would otherwise have been written three times.

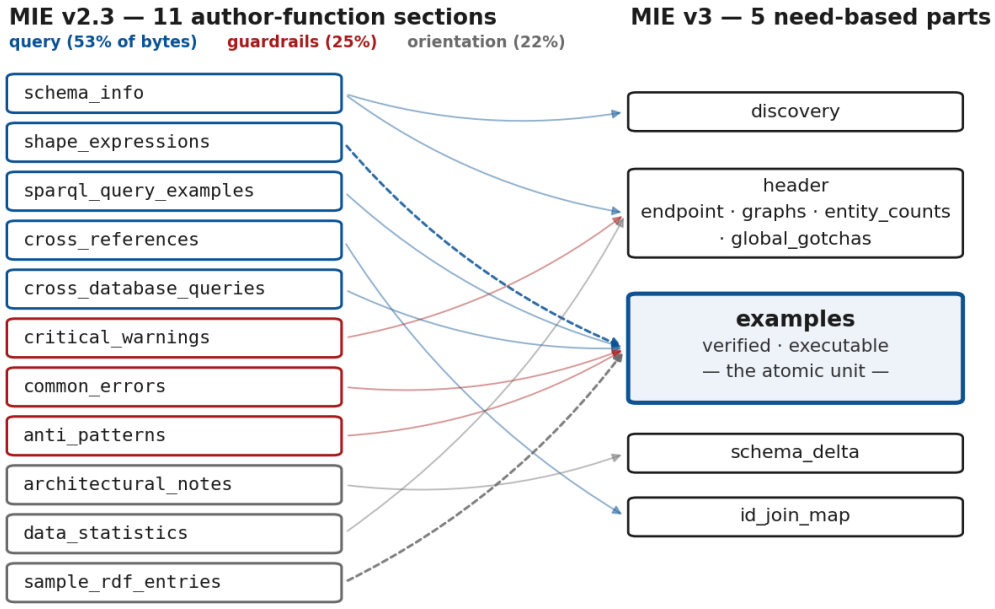

Figure 2: The v2.3 format's eleven author-function sections mapped onto v3's five need-based parts. `shape_expressions` and `sample_rdf_entries` do not survive as sections at all: a verified example subsumes both.

Six authoring rules carry the design, each with a reason:

- **§4.1 Everything countable is verified and dated.** Every entity count and every example result is re-run live against the endpoint and carries the date it was run. This makes the file machine-testable: CI can execute every example and assert its recorded result, and a disagreement becomes a drift signal rather than silent rot. (A YAML footgun, worth passing on: the bare key `on:` parses as a boolean under YAML 1.1, so a validator looking for `on` finds nothing. Use `date:`, and quote the value. The v3 format has its own literal-typing traps, exactly like the SPARQL ones the MIEs document.)
- **§4.2 One fact, one place.** A warning is either database-wide or query-specific, never both.
- **§4.3 Carry only the non-recoverable.** If the model can get it from training or one exploratory `SELECT`, cut it. An example's own scaffolding rides for free, because the non-recoverable idiom cannot be shown without it.
- **§4.4 A positive route is not a caveat.** Discussed below; this rule was bought with a regression.
- **§4.5 Progressive disclosure.** The header is authored to stand alone, so a future `get_MIE_file(database, level=...)` can serve tiers.
- **§4.6 Illustrative subjects must not be drawn from the benchmark.** Also discussed below.

The resulting corpus is 36 files containing **303 examples**, each **29–65% smaller** than the v2 file it replaces. Two files were hand-authored as pilots and 34 were delegated one agent per database, each independently re-validated by the caller rather than trusted on the builder's own check. Live verification during authoring caught real errors in the *v2* corpus along the way — a mislabelled ChEBI role IRI, a dropped Rhea cross-join, a PDB EC-prefix over-match, a ClinVar gene-blank-node split, among others. That is a return on "verified and dated" quite separate from the token budget.

Validation: smoke, canary, then the full equivalence run

The release gate was declared in advance as an **equivalence** test, not as "did the score go up?" Three criteria, fixed before any data was collected: bytes down (deterministic, no statistics required), judge score non-regressing against a **-0.5/20** margin, and the factoid judge score up-or-flat. Framing it this way mattered — with a ceiling-limited benchmark, a redesign whose purpose is compression should be *asked* to prove non-inferiority, not allowed to fish for an improvement.

The smoke test bought us a rule

Before authoring 36 files we swapped two, and ran the 25 benchmark questions in which one of them is one of exactly two databases — the subset where a swapped MIE accounts for half the database content, and therefore gives the strongest per-question signal. The average was flat (-0.44 $\pm$ 0.82; the UniProt slice -0.13 over 20 questions), just inside the declared bail bar and so a yellow light rather than a green one — and it decomposed non-randomly, with the multi-step types regressing (factoid -1.09, list -1.04) while the easier ones improved. Underneath sat one systematic regression. On q066 — reviewed human proteins with a LIM domain — v3 retrieved 14 candidate proteins against the true 71, and named the wrong winner on all three replicates.

The diagnosis generalizes. UniProt keyword classification (`up:classifiedWith keywords:NNN`) is *the* route for enumerating all proteins with a given feature; the v2 file documented it as a first-class route in about five places. The v3 draft compressed all of that into a single *negative caveat* attached to a different example — "`up:classifiedWith` also carries keywords, filter them out" — so the agent read the route as noise to exclude and fell back to text matching, which silently undercounts. Hence **§4.4: a positive route is not a caveat.** Many mechanisms are dual, being both "the way to do X" and "watch out for X when doing Y", and compression preferentially keeps the vivid warning and discards the query.

The epilogue is worth reporting because it nearly went wrong. Our first fix used q066's *exact* subject — LIM domain — and even carried the intermediate count, 71, that the correct route produces at its first step. That made the recovery partly circular: the corpus now contained a waypoint on the path to the answer. We swapped the illustrative subject to a neutral one (SH3 domain) and re-ran: the agent generalized the idiom to LIM domain and scored 18 on all three replicates, with no LIM entity anywhere in the corpus. The de-overfit version outperformed the overfit one. (This targeted three-replicate re-run was the one measurement we took under subscription rather than API authentication — see Trap 6 — but at this size the signal is unambiguous.) That produced **§4.6**, the no-test-leakage rule, which matters generally now that documentation and evaluation sets are both routinely authored with model assistance.

The canary caught a subtler failure

A ten-question risk-first canary flagged q022, where v3 scored 5.7 points below v2. The cause was not a lost route. The GlyCosmos endpoint hosts **its own partial snapshot of the Gene Ontology**, and v3 was querying it self-consistently: its `subClassOf*` closure yields 14 terms and therefore 44 genes. The correct answer requires expanding on the authoritative go database — 33 terms — and joining the result back, giving 208, exactly the gold answer. We added a first-class two-database enumeration example; q022 moved from -5.7 to **+1.0**, and v3 became *more* stable than v2, which had reached 208 only by calling an external ontology service and collapsed to 44 on one of its own runs. The general lesson: **a co-hosted ontology snapshot is not the ontology**, and an agent has no way to discover that on its own.

The full run

The final gate ran all 100 questions × 3 replicates × 2 corpora — the ten-question canary plus five further batches of 25, 15, 25, 10 and 15, with a review gate after each.

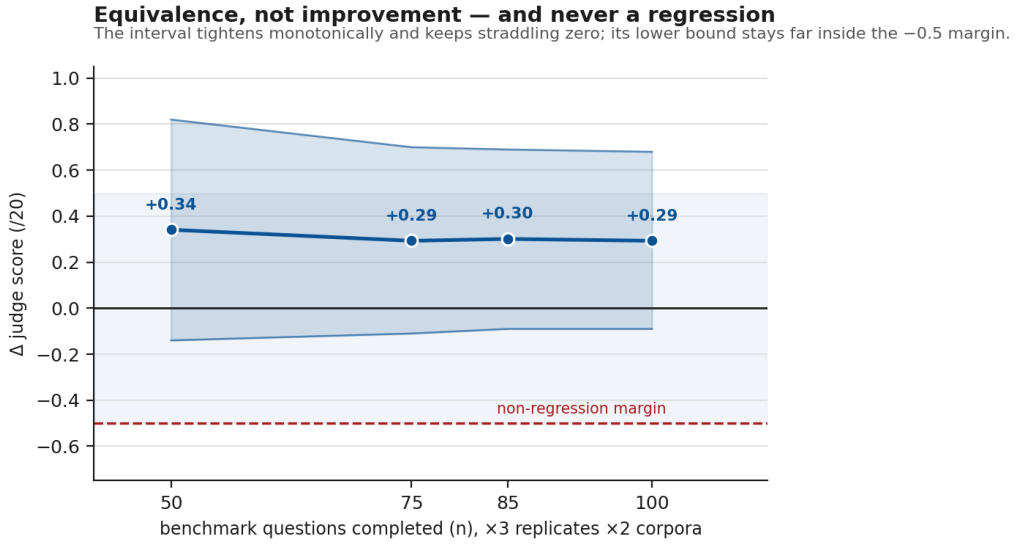

Figure 3: The equivalence ladder. The point estimate is stable and the interval tightens monotonically as questions accumulate, while continuing to straddle zero; the lower bound remains far inside the pre-declared non-regression margin.

At n=100, the refusal-clean paired difference is **+0.293/20 with a 95% CI of [-0.09, +0.68]** across 96 usable questions; alternative estimators agree (pooled +0.343, strict-all-clean +0.368), and the per-question tally is 36 better, 33 tied, 27 worse. The interval straddles zero, so this is **genuine equivalence and not a proven gain** — the mild positive tilt should not be over-claimed. What the data does support is the non-regression conclusion: a lower bound of -0.09 sits far inside the declared -0.5 margin.

By question type, the pattern matches what v3 was designed to do:

Table 5: Change in judge score by question type at n=100 (refusal-clean).

Type	Change (v3 - v2)
factoid	**+1.01**
yes_no	+0.54
list	+0.18
choice	+0.16
summary	-0.42

Factoid questions — the query-construction and aggregation cases the executable worked examples target directly — gain a full point, which discharges the third gate criterion. The recall sub-score rises +0.16 overall, corroborating that v3 retrieves the right facts at least as often. The single soft spot is `summary`, where v3's terseness offers less scaffolding for open-ended prose synthesis; several of those questions are shared blind spots that trip v2 equally.

The deterministic win, measured at runtime rather than inferred from file sizes:

Table 6: Per-question runtime cost at n=100 (279 clean v2 cells and 282 clean v3, of 300 each).

Metric (per question)	v2	v3	Change
total input tokens	73,059	61,790	**-15.4%**
· cache-read	71,394	59,255	-17.0%
· cache-creation	1,658	2,527	+52.4%
output tokens	656	686	+4.6%
cost (USD)	0.52	0.44	**-14.8%**
wall time (s)	150.9	142.0	**-5.9%**

A 29–65% reduction in file size lands as a 15% reduction in *total* input tokens because the MIE is a large but partial share of per-question context — system prompt, tool schemas, SPARQL result payloads and accumulated reasoning all persist alongside it. The dominant term is cache-read, which is precisely the recurring cost: a smaller document is re-read from cache on every turn of every session. On this run alone the v3 arm cost about US$20 less than the v2 arm.

One data-quality caveat, stated plainly: spurious content-policy refusals contaminated 6.3% of cells, slightly more of the v2 arm than of v3 (21 against 17 of 300 each), and rendered four questions unmeasurable. Those are excluded from the clean verdict; the equivalence conclusion rests on the 96 measurable questions.

Discussion

Redundancy is invisible to leave-one-out. This is the finding we would most want carried elsewhere. "Not individually necessary given everything else" is not "worthless," and a null leave-one-out result licenses neither conclusion on its own. Distinguishing them requires total removal, to establish that there is any value at all, and leave-one-in, to locate it. Our four families only tell a coherent story as a set: null, null, significant, sufficient.

The corollary is a budgeting one, and it is the single thing we would change about how we ran this. Those two decisive families are also the *cheap* ones — four conditions between them against eleven for the leave-one-out sweep that answered nothing. We spent the most money on the least informative experiment because leave-one-out is the reflexive first move, and we did it before we had any estimate of the total effect it was trying to decompose. Measure the whole first; it costs one condition, and it tells you whether the decomposition you were planning is affordable or arithmetically hopeless.

Failure-driven documentation accretes redundancy, and cannot notice. Our eleven sections each came from a real observed failure, which is why the corpus works — and also why it triplicated. A given failure can be answered in several places, each answer is locally correct, and nothing in the process ever revisits whether an earlier answer already covered it. The accumulation is monotone: sections are added when something breaks, and removed never, because removing one has no visible consequence — which is precisely what our leave-one-out results confirm. Any project that documents reactively should expect this, and should expect that its own null ablations will be read, wrongly, as evidence that the content is dead.

The verified example is the right atomic unit for LLM-facing schema documentation. A single executable query carries the shape, an instance, and the trap simultaneously, in the form the agent will actually need them, and — unlike prose — it can be executed and checked. This is the design claim we would defend beyond TogoMCP.

We have modest evidence that it already travels. One of the 36 databases in the corpus is not a life-science resource at all: SuperCon, the NIMS superconducting-materials database, whose `discovery` categories are *materials* and *physics* where every other file reads as biology.

Its MIE file has the same five parts, nine examples all verified and dated, and a first-class set-level enumeration example exactly as §4.4 requires. The one structural difference is that it carries no `cross_db` example — not because the format could not express one, but because the database has nothing to join to. That is the format reporting a fact about the resource rather than bending to it.

A single case is not a demonstration. But the rules the ablations motivated are stated in terms of an agent's need for query-construction context and the recoverability of a given fact, neither of which is specific to biology — or, for that matter, to RDF. The obvious next test is whether the same measure-then-redesign loop reproduces on a structured resource of an entirely different shape.

Compression preferentially destroys positive routes. The q066 failure has a general shape: a mechanism that is both the way to do something and a hazard when doing something else survives compression as its warning, which reads to an agent as *avoid this*. Anyone summarizing documentation — by hand or, increasingly, with a model — should expect this failure mode and check for it explicitly.

Documentation becomes a testable artifact. Requiring every countable value to be verified and dated changes the maintenance economics of a 36-database corpus more than the token saving does. Drift stops being silent. In our case the authoring pass caught genuine errors in the corpus it was replacing.

Agent-authored documentation needs a leakage rule. When the docs and the evaluation set are both produced with model assistance, an example can quietly contain the test answer, and the resulting benchmark improvement is real-looking and worthless. A rule plus a grep against the question set is a cheap general guard.

Limitations. A single answering model and a single judge family; one benchmark, whose ceiling of roughly 17/20 caps the effect sizes that can be resolved; a 40-question pilot for the ablations, where our best near-miss would have needed n of about 73; LLM-as-judge scoring, with its known biases; 6.3% refusal contamination; and an equivalence result that is equivalence, not superiority. We also measured a single format against a single alternative — v3 is evidence that *this* reorganization preserves value at fewer tokens, not that it is optimal.

Future work

The immediate item is **progressive disclosure**: the v3 header was deliberately authored to stand alone, so `get_MIE_file` can serve a cheap tier and an expensive one. Next, **CI that executes every example's recorded result** against the live endpoint, turning drift into a build failure. Third, **per-query outcome logging** — recording whether each `run_sparql` call returned a syntax error, an empty result, or rows — which would be a far sharper effort metric than counting calls, and a direct test of whether query-guidance content does what it claims. For statistical power, the variance decomposition says to buy precision with answer replicates rather than judge replicates, and to make exact-answer correctness the primary endpoint at n of 150 or more. Finally, cross-model replication, and applying the same measure-then-redesign loop to the other large piece of LLM-facing text in the system, the Usage Guide.

Availability

TogoMCP is at https://togomcp.rdfportal.org/ and its source, including the ablation harness, the 100-question benchmark, and the durable findings records this report draws on, at https://github.com/dbcls/togomcp. The MIE v3 corpus and format specification ship in release **v2.0.0**, which flipped the served corpus to v3 and retired the now-redundant database-discovery tools.

That release is archived on Zenodo as a citable snapshot, under CC-BY 4.0: doi:%5B10.5281/zenodo.21543297](https://doi.org/10.5281/zenodo.21543297) (Yamamoto & Kinjo, 2026). It contains the full v3 corpus, the MIE v3 specification, the ablation and equivalence harnesses, the 100-question benchmark, and the `FINDINGS.md` records from which every number in this report is drawn.

This report and its sources are at https://github.com/arkinjo/mie-redesign.

Acknowledgements

This work was carried out at **Togothon**, the monthly knowledge-graph development meeting organized by DBCLS (formerly SPARQLthon), and specifically at Togothon 166, 23–24 July 2026. As Togothon is not currently a registered BioHackrXiv meeting, this report is filed under the DBCLS BioHackathon 2025 (BH25JP), at which the MIE format it revises was formalized.

We thank the Togothon community for discussion, Jose-Emilio Labra Gayo for naming the MIE format at the DBCLS BioHackathon 2025, and the maintainers of the RDF Portal endpoints whose data made the benchmark possible.

References

Anthropic. (2025). *Model Context Protocol specification, revision 2025-11-25*. https://modelcontextprotocol.io/specification/2025-11-25. **[cito:citesAsAuthority]**

Emonet, V., Bolleman, J., Duvaud, S., Mendes de Farias, T., & Sima, A. C. (2024). LLM-based SPARQL query generation from natural language over federated knowledge graphs. *arXiv Preprint*. https://doi.org/10.48550/arXiv.2410.06062 **[cito:citesAsAuthority]**

Ikeda, S., Ono, H., Ohta, T., Chiba, H., Naito, Y., Moriya, Y., Kawashima, S., Yamamoto, Y., Okamoto, S., Goto, S., & Katayama, T. (2022). TogoID: An exploratory ID converter to bridge biological datasets. *Bioinformatics*, *38*(17), 4194–4199. https://doi.org/10.1093/bioinformatics/btac491 **[cito:citesForInformation]**

Kawashima, S., Katayama, T., Hatanaka, H., Kushida, T., & Takagi, T. (2018). NBDC RDF portal: A comprehensive repository for semantic data in life sciences. *Database*, *2018*, bay123. https://doi.org/10.1093/database/bay123 **[cito:usesDataFrom]**

Kinjo, A. R., Yamamoto, Y., Bustamante-Larriet, S., Labra-Gayo, J.-E., & Fujisawa, T. (2026). TogoMCP: natural language querying of life-science knowledge graphs via schema-guided LLMs and the Model Context Protocol. *Database*, *2026*, baag042. https://doi.org/10.1093/database/baag042 **[cito:citesAsAuthority]**

Labra-Gayo, J. E., Yamamoto, Y., Kinjo, A. R., Waagmeester, A., Millán Acosta, J., Kawashima, S., Okabeppu, Y., Koblitz, J., Bustamante Larriet, S., & Fernández-Álvarez, D. (2025). MCP server tools with RDF shapes. *BioHackrXiv*. https://doi.org/10.37044/osf.io/8qeh5_v1 **[cito:extends]** **[cito:citesAsSourceDocument]**

Tsatsaronis, G., Balikas, G., Malakasiotis, P., Partalas, I., Zschunke, M., Albers, M. R., Weissenborn, D., Krithara, A., Petridis, S., Polychronopoulos, D., Almirantis, Y., Pavlopoulos, J., Baskiotis, N., Gallinari, P., Artières, T., Ngomo, A.-C. N., Heino, N., Gaussier, E., Barber, L., … Paliouras, G. (2015). An overview of the BioASQ large-scale biomedical semantic indexing and question answering competition. *BMC Bioinformatics*, *16*, 138. https://doi.org/10.1186/s12859-015-0564-6 **[cito:usesMethodIn]**

Yamamoto, Y., & Kinjo, A. R. (2026). *TogoMCP: Natural Language Querying of Life-Science Knowledge Graphs via Schema-Guided LLMs and the Model Context Protocol (GitHub repo snapshot, v2.0.0)* (Version v2.0.0) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.21543297 **[cito:citesAsDataSource]**

Zahera, H. M., Sherif, M. A., & Ngonga Ngomo, A.-C. (2024). Generating SPARQL from natural language using chain-of-thoughts prompting. *Proceedings of the 20th International Conference on Semantic Systems (SEMANTiCS 2024)*, *60*, 353–368. https://doi.org/10.3233/SSW240028 **[cito:citesAsAuthority]**

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, *36*. https://doi.org/10.48550/arXiv.2306.05685 **[cito:usesMethodIn]**

Author contributions

Akira R. Kinjo: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft Yasunori Yamamoto: Conceptualization, Resources, Writing – review & editing